

ML-Driven Archaeological Site Prediction Plugin for QGIS

Architecture, Data Pipeline, Model Selection & Implementation Roadmap

April 2026 | Version 1.0 | For Computational Archaeologists

Executive Summary

This report defines the technical specification for an open-source QGIS plugin that enables computational archaeologists to build, validate, and interactively update machine learning models for predictive archaeological site prospection — without requiring Python programming expertise. The plugin bridges a persistent gap: ML methods for site prediction are well-established in academic literature but remain locked inside bespoke scripts, inaccessible to most field practitioners.

The proposed plugin — provisionally named ArchPredict — integrates directly into the QGIS Processing Framework and provides a no-code workflow from environmental raster ingestion through model training, probability surface generation, cross-validation, and iterative refinement as new site data is collected.

Core value proposition: A field archaeologist with no ML background should be able to produce a calibrated, validated site probability map from existing survey data within a single QGIS session.

1. Background & Problem Statement

1.1 The Prospection Challenge

Archaeological survey and excavation are expensive, time-consuming, and destructive by nature. Selecting where to look is one of the most consequential methodological decisions in

field archaeology. Predictive site location modeling — using known site distributions and environmental covariates to forecast where undiscovered sites are likely — has been practiced since the 1980s, but the tooling situation has barely improved for practitioners.

The standard workflow today involves: (1) manual extraction of environmental variables from GIS, (2) export to a statistics package (R, SPSS, or Python), (3) model fitting, (4) probability surface generation, (5) manual re-import to GIS. This pipeline is fragile, non-reproducible, and excludes the majority of working archaeologists who lack programming skills.

1.2 What the Literature Shows

Recent reviews confirm that ML approaches — particularly Random Forest, Gradient Boosting, and MaxEnt — substantially outperform traditional logistic regression for site prediction, achieving AUC scores of 0.80–0.95 on well-curated datasets. However, the same reviews note:

- Most published models are never reused or shared after the paper
- Validation practices are inconsistent and often methodologically weak
- The absence of uncertainty quantification leads to overconfident predictions
- No standard data schema exists for archaeological site inventories in GIS

1.3 Existing QGIS Plugin Landscape

A survey of the QGIS plugin repository reveals the following coverage in the archaeology/landscape analysis space:

Plugin	Coverage	ML Capability
ArchToolkit	Viewshed, cost surface, terrain	None
GeoArchaeo	Kriging, variogram, PCA	Basic pattern recognition
pyArchInit	Site/find cataloguing, stratigraphy	None
AGT	Geophysics data processing	None
ArchaeoloGIS	Road network analysis	None

No existing plugin provides an end-to-end predictive modeling workflow. The gap is not incremental — it is categorical.

2. Technical Architecture

2.1 Integration Strategy: QGIS Processing Framework

The plugin must integrate with the QGIS Processing Framework (QgsProcessingAlgorithm) rather than implementing a standalone GUI. This is the architecturally correct choice for three reasons:

1. Processing algorithms appear in the Processing Toolbox, the Graphical Modeler, and can be called from the QGIS Python console — giving users multiple interaction modes for free
2. The framework handles input/output layer management, progress reporting, and cancellation automatically
3. Algorithms can be chained into reproducible Processing Models that encode the full analytical pipeline as a shareable file

The plugin registers a custom Processing Provider (QgsProcessingProvider) that exposes individual algorithms for each stage of the workflow. This makes the plugin composable: users can run only the stages they need, or plug ArchPredict algorithms into existing models.

2.2 Module Structure

The plugin is organized into five functional modules:

Module	Responsibility	Key Classes
archpredict/core	Model training, prediction, CV	ModelEngine, CrossValidator
archpredict/data	Raster/vector ingestion, sampling	FeatureExtractor, Sampler
archpredict/processing	QGIS Processing algorithms	TrainAlgorithm, PredictAlgorithm
archpredict/ui	Custom panels, result widgets	ResultDock, ValidationPanel
archpredict/io	Model persistence, export	ModelSerializer, ReportExporter

2.3 Dependency Management

The principal challenge in QGIS plugin development is managing Python dependencies without conflicting with QGIS's own Python environment. The recommended approach for ML-heavy plugins:

Use a plugin-local virtual environment (venv) isolated inside the plugin directory. On first run, the plugin checks for required packages and offers to install them automatically via pip into this venv. This avoids the OSGeo4W environment on Windows and the system Python on Linux/macOS.

Required Python packages and their roles:

- scikit-learn >= 1.4: Random Forest, Gradient Boosting, MaxEnt proxy (OneClassSVM), cross-validation utilities
- numpy >= 1.26: Array operations for raster manipulation
- pandas >= 2.0: Tabular data handling for training datasets
- rasterio >= 1.3: Direct raster I/O (faster than GDAL Python bindings for bulk reads)
- joblib >= 1.3: Parallel processing and model serialization
- matplotlib >= 3.8: In-plugin charts (ROC curves, feature importance)
- shap >= 0.44: SHAP values for model explainability (optional but recommended)

3. Data Pipeline

3.1 Input Data Requirements

The plugin accepts two categories of inputs:

3.1.1 Site Presence Data

A vector point or polygon layer representing known archaeological sites. Required attributes:

- Geometry: Point or polygon (centroid extracted automatically for polygons)
- Optional: site_type field for stratified analysis by site category
- Optional: period field for temporal filtering

The plugin validates the CRS of all inputs and reprojects to a common CRS (user-selected) at the start of each run.

3.1.2 Environmental Predictor Rasters

Any number of single-band raster layers representing environmental covariates. Archaeologically relevant predictors typically include:

Predictor Category	Example Variables	Source
Topography	Elevation, slope, aspect, TPI, TRI, curvature	DEM derivatives (GDAL/SAGA)
Hydrology	Distance to water, flow accumulation, TWI	DEM hydrological analysis
Soil	Soil type, drainage class, pH, texture	National soil surveys
Visibility	Viewshed score, intervisibility index	QGIS viewshed tools
Land cover	NDVI, historical land use class	Remote sensing products
Accessibility	Least-cost distance, path proximity	Cost surface analysis

Predictor Category	Example Variables	Source
Geology	Lithology class, bedrock depth	Geological surveys

3.2 Pseudo-Absence Generation

Most archaeological datasets contain only presence records (known sites) with no confirmed absences. This is the central methodological challenge of the field and is the leading source of model bias. The plugin implements three strategies:

Strategy A: Random Background Sampling

Generate random background points across the study area, treating them as pseudo-absences. This is the MaxEnt approach and is appropriate when survey coverage is uniform.

```
n_background = n_sites * background_ratio # default ratio: 10:1
background = random_sample(study_area_mask, n=n_background, seed=random_state)
```

Strategy B: Environmentally Stratified Sampling

Sample background points such that their environmental distribution matches the full landscape distribution. This reduces the confounding effect of environmental bias in survey effort.

```
# Bin each predictor into quantiles, sample proportionally from each bin
background = stratified_env_sample(predictors, study_area, n=n_background)
```

Strategy C: Target-Group Sampling (Survey-Effort-Corrected)

If a survey effort raster is available (e.g., distance from survey transects), weight background sampling to match the spatial bias of the presence data. This is the most statistically rigorous approach and is recommended when data comes from non-systematic surveys.

Implementation note: The plugin exposes all three strategies as parameters in the Training Algorithm dialog, with clear documentation of when each is appropriate. The default is Strategy B, which provides a reasonable bias correction with no additional data requirement.

3.3 Feature Extraction

For each presence and background point, the plugin extracts predictor values from all raster layers. The extraction process:

4. Checks raster alignment (resolution, extent, CRS) — misaligned rasters are resampled to a user-specified reference grid using bilinear resampling for continuous variables and nearest-neighbor for categorical
5. Handles NoData values: rows with any NoData predictor are dropped from training, flagged in prediction output

6. Applies optional spatial filtering: a minimum nearest-neighbor distance parameter prevents spatial autocorrelation in training data (removes duplicate detections at the same site)
7. Outputs a pandas DataFrame with columns: [point_id, x, y, presence, pred_1, pred_2, ..., pred_n]

4. Model Selection & Rationale

4.1 Recommended Primary Model: Random Forest

Random Forest (RF) is the recommended default algorithm for ArchPredict based on its demonstrated performance in the archaeological literature and its practical properties:

- Consistently achieves AUC 0.80–0.92 in archaeological site prediction benchmarks
- Handles mixed predictor types (continuous, categorical, ordinal) without preprocessing
- Produces native feature importance scores that are archaeologically interpretable
- Robust to correlated predictors — common in environmental datasets — without requiring prior variable selection
- Does not require feature scaling, reducing a common source of user error
- Training is parallelizable (n_jobs=-1) making it practical on consumer hardware

4.1.1 Hyperparameter Defaults & Tuning

The plugin exposes the following RF hyperparameters in the Advanced Parameters section of the Training Algorithm:

Parameter	Default	Range	Archaeological Guidance
n_estimators	500	100–2000	Higher = more stable; 500 sufficient for most datasets
max_features	'sqrt'	sqrt / log2 / float	sqrt generally optimal for classification
max_depth	None	5–50 / None	None (full growth) unless dataset is small (<500 sites)
min_samples_leaf	5	1–50	Increase to 10–20 for small, sparse site inventories
class_weight	'balanced'	balanced / dict	Always use 'balanced' for presence/background data
random_state	42	Any integer	Set for reproducibility; document in output metadata

4.2 Secondary Models

The plugin also implements two additional algorithms selectable from the model dropdown:

4.2.1 Gradient Boosted Trees (XGBoost / scikit-learn HistGBT)

Often matches or slightly outperforms Random Forest on larger datasets (>1,000 sites). Slower to train, more sensitive to hyperparameters, but produces tighter probability estimates. The plugin uses scikit-learn's HistGradientBoostingClassifier (no XGBoost dependency) for installation simplicity. Recommended for regional-scale studies with dense site inventories.

4.2.2 MaxEnt Proxy (Regularized Logistic Regression on Environmental Background)

Implements the MaxEnt modeling philosophy using scikit-learn's LogisticRegression with L2 regularization and feature expansion (polynomial and interaction terms up to degree 2). This mirrors the approach of the MaxEnt software without requiring the Java dependency. Best for presence-only scenarios where the background-to-presence ratio is high (>20:1) and interpretability of individual predictor contributions is prioritized.

Model selection guidance shown in plugin UI: A simple flowchart widget guides users through algorithm choice based on dataset size, data type (presence-only vs. presence/absence), and computational constraints.

5. Validation Framework

5.1 The Spatial Autocorrelation Problem

Standard k-fold cross-validation is methodologically invalid for spatial prediction models. Because archaeological sites cluster spatially, a randomly split test set will contain points geographically close to training points — artificially inflating performance metrics. The plugin enforces spatially-aware validation by default.

Critical: A model validated with random k-fold CV on spatially clustered data routinely reports AUC 0.10–0.25 higher than its true predictive performance in new survey areas. This is not a minor correction.

5.2 Spatial Block Cross-Validation

The plugin implements spatial block CV as the default validation strategy:

8. The study area is divided into a regular grid of blocks. Block size is a tunable parameter, defaulting to 5x the mean nearest-neighbor distance between known sites

9. Each CV fold assigns one or more complete spatial blocks to the test set, ensuring geographic separation between train and test
10. The plugin optionally implements buffered exclusion zones: training points within a buffer distance of test blocks are excluded, further reducing spatial leakage
11. 5-fold spatial block CV is performed, with mean and standard deviation of AUC, TSS, and Boyce Index reported across folds

5.3 Validation Metrics

The plugin reports a comprehensive set of validation metrics in the Results Panel:

Metric	Formula / Method	Interpretation
AUC-ROC	Area under ROC curve	Overall discrimination ability; 0.5 = random, 1.0 = perfect
TSS	Sensitivity + Specificity - 1	Threshold-dependent; >0.5 considered useful
Boyce Index	Spearman r of predicted/expected ratio	Specifically designed for presence-background models
Omission Rate	False negatives / total positives	Fraction of known sites in low-probability areas
Gain over random	% study area classified high vs. % sites captured	Core practical metric for survey planning

5.4 Null Model Testing

The plugin implements a permutation-based null model test: the model is retrained 99 times with randomized site locations, generating a null distribution of AUC scores. The observed model AUC is compared to this distribution to compute a p-value. This test confirms that observed model performance is not simply a product of spatial autocorrelation in the predictors.

6. Output Products

6.1 Probability Surface Raster

The primary output is a continuous raster (GeoTIFF) of site occurrence probability at each pixel, ranging 0–1. The plugin generates this by applying the trained model to the full predictor stack across the study area extent. Pixels with NoData in any predictor layer receive NoData in the output.

Optionally, a second raster of prediction uncertainty is produced by computing the standard deviation of individual tree predictions within the Random Forest ensemble. High uncertainty areas (where trees disagree) are flagged as requiring ground-truthing priority.

6.2 Classified Priority Map

A classified version of the probability surface is produced using three default thresholds (low / medium / high priority), with the threshold values derived from the ROC analysis at maximum TSS. The plugin allows manual threshold adjustment via an interactive slider in the Results Dock, with real-time reclassification of the priority map.

6.3 Feature Importance Chart

A ranked bar chart of predictor importance is displayed in the Results Dock and exported as a PNG. For Random Forest, this uses the mean decrease in impurity (MDI) metric. When SHAP is installed, SHAP summary plots are generated instead, providing more reliable importance rankings and directional effects.

6.4 Model File

The trained model is serialized to a .archpredict file (a ZIP archive containing the joblib-serialized model, metadata JSON, and predictor layer registry). This file can be shared, reloaded in a new QGIS session, or applied to a new study area using the Apply Existing Model algorithm.

6.5 Reproducibility Report

A structured HTML report is generated summarizing: software versions, input layer hashes, all hyperparameter values, cross-validation results, and the random state used. This report is designed to meet the reproducibility standards expected by journals such as the Journal of Computer Applications in Archaeology (JCAA).

7. Implementation Roadmap

7.1 Phase 1 — Foundation (Months 1–3)

- Set up QGIS plugin scaffold with Processing Provider registration
- Implement dependency management (venv creation, pip install on demand)
- Build FeatureExtractor: multi-raster sampling with CRS alignment and NoData handling
- Implement pseudo-absence generator (Strategies A and B)
- Implement Random Forest training algorithm with basic UI
- Generate probability raster output
- Basic validation: random k-fold CV (to be superseded in Phase 2)
- Unit tests for data pipeline using synthetic rasters

7.2 Phase 2 — Validation & Robustness (Months 4–6)

- Implement spatial block cross-validation
- Add buffered exclusion zones in CV
- Add TSS, Boyce Index, and permutation null test
- Implement prediction uncertainty raster (RF ensemble variance)
- Add Gradient Boosted Trees and MaxEnt proxy algorithms
- Implement model serialization and Apply Existing Model algorithm
- Survey effort raster support (Strategy C pseudo-absence)
- Integration tests against published benchmark datasets

7.3 Phase 3 — UX & Explainability (Months 7–9)

- Results Dock with interactive threshold slider
- Feature importance charts (MDI and optional SHAP)
- Model selection flowchart widget
- Reproducibility HTML report export
- Localization scaffold (EN/DE/IT/ES as first targets)
- Full user documentation and worked tutorial with sample data
- QGIS Plugin Repository submission and peer review

7.4 Phase 4 — Advanced Features (Months 10–12+)

- Iterative refinement: add new site records and retrain incrementally
- Ensemble model stacking across multiple algorithms
- Temporal stratification: train period-specific models and compare
- Integration with ARIADNE and Open Context APIs for federated site data
- Optional LiDAR-derived predictor computation (integrated with PDAL)

8. Key Technical Risks & Mitigations

Risk	Severity	Mitigation
QGIS Python env conflicts on Windows (OSGeo4W)	High	Isolated plugin venv; bundled wheels for scikit-learn, numpy
Small site inventories (<50 sites) causing model overfit	High	Enforce min_samples_leaf=10; warn user; suggest leave-one-out CV
User misinterpretation of probability as ground truth	High	Mandatory uncertainty raster; prominent UI warnings; documentation
Raster alignment failures on heterogeneous input stacks	Medium	Auto-resampling to reference grid; pre-flight alignment checker

Risk	Severity	Mitigation
Performance on very large study areas (>10M pixels)	Medium	Chunked raster prediction; progress bar; optional downsampling
Survey bias producing spatially biased models	Medium	Expose Strategy C sampling; include bias layer parameter

9. Minimum Dataset Requirements

The plugin enforces the following minimums and displays appropriate warnings:

- Minimum 30 known site locations for model training (warning below 30, hard stop below 10)
- Minimum 3 environmental predictor rasters (model with fewer predictors is unlikely to generalize)
- Study area extent must be at least 4x the convex hull of known sites (to capture true environmental range)
- No predictor should have >30% NoData within the study extent

Note on data scarcity: When site inventories are small, the plugin will recommend switching from spatial block CV (which requires sufficient blocks) to leave-one-out spatial CV, and will increase regularization defaults automatically.

10. Ethical & Methodological Considerations

Predictive site models carry specific ethical responsibilities in archaeology that must be communicated in the plugin interface and documentation:

- Models trained on known sites inherit the biases of past survey coverage. Areas with low survey intensity will appear low-probability even if genuinely rich. The plugin makes this explicit in the output metadata.
- High-probability outputs should never be interpreted as confirmation of site existence — they are survey prioritization tools, not discovery claims.
- In Cultural Resource Management (CRM) contexts, model outputs may influence decisions about development permits and site protection. The reproducibility report is specifically designed to support regulatory documentation requirements.
- The plugin must not be used to identify specific site locations for non-archaeological purposes. The documentation includes a responsible use statement.

11. Summary: Recommended Technology Stack

Component	Technology	Notes
Plugin framework	QGIS 3.34+ / PyQGIS	LTR version as minimum target
ML engine	scikit-learn 1.4+	No heavy dependencies; well-maintained
Raster I/O	rasterio + GDAL	rasterio for bulk read; GDAL for format support
Data wrangling	numpy + pandas	Standard scientific Python stack
Serialization	joblib	Efficient model persistence; included with sklearn
Visualization	matplotlib	In-plugin plots; no browser dependency
Explainability	SHAP (optional)	Optional install; graceful fallback to MDI
Testing	pytest + pytest-qgis	QGIS-aware test runner
CI/CD	GitHub Actions	QGIS Docker images available for automated testing
Distribution	QGIS Plugin Repository	Standard distribution channel

References & Further Reading

The following works directly inform the methodological choices in this report:

- Verhagen, P. et al. (2020). Predictive Modeling in CRM: Challenges and Solutions. *Journal of Computer Applications in Archaeology*.
- Bickler, S.H. (2021). Machine Learning Arrives in Archaeology. *Advances in Archaeological Practice*.
- Roberts, D.R. et al. (2017). Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*. [Spatial block CV methodology]
- Boyce, M.S. et al. (2002). Evaluating resource selection functions. *Ecological Modelling*. [Boyce Index metric]
- Hijmans, R.J. (2012). Cross-validation of species distribution models: removing spatial sorting bias and calibration with a null model. *Ecology*. [Null model testing]
- Lundberg, S.M. & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NeurIPS*. [SHAP values]
- Cardarelli, A. (2025). Machine learning in archaeology: a systematic review. *Journal of Computer Applications in Archaeology*.

This report is intended as a technical foundation document for the development of ArchPredict. All architecture decisions are subject to revision following community consultation with the CAA (Computer Applications and Quantitative Methods in Archaeology) working group.